Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Уфимский государственный нефтяной технический университет»

Кафедра «Цифровые технологии и моделирование»

09.04.01 Информатика и вычислительная техника

Разработка нейроадаптивной VR-игры с динамической сложностью на основе уровня стресса пользователя

Курсовая работа

Работа защищена:

« »_____2026 г.

Оценка_____
(подпись)

Выполнил студент
2 курса МПИ09-25-01 группы

Гильмуханов Д.И.

_____
(ф.и.о., подпись)

Научный руководитель

_____
(уч. Степень, уч.звание, фамилия, инициалы)

_____
(подпись)

Уфа 2026

СОДЕРЖАНИЕ

Введение……..…..3

1 Анализ предметной области и технологий нейровиртуальных систем…….....5

1.1 Анализ предметной области и существующих решений……....5

1.2 Анализ методов и технологий реализации…….……..7

1.3 Постановка задачи и функциональные требования…….……8

2 Проектирование и реализация адаптивной VR-Игры…….10

2.1 Выбор и обоснование средств реализации…….10

2.2 Проектирование архитектуры и алгоритмов……..11

2.3 Программная реализация и проверка функционирования……13

Заключение…….19

Список использованных источников……...21

Приложение А. Код сервиса классификации…….….23

Приложение Б. Код скрипта Unity……...…24

ВВЕДЕНИЕ

Виртуальная реальность используется для создания игровых и учебных сред, где действия человека изменяют происходящее на сцене. Сложность в таких приложениях нередко задаётся заранее либо зависит от результатов прохождения. Однако успешность выполнения задания не всегда отражает текущее состояние игрока: одинаковая нагрузка может восприниматься по-разному. Поэтому для адаптации игры рассматривается дополнительный источник данных – психофизиологические показатели пользователя.

Цель курсовой работы – разработать нейровиртуальный программный комплекс, способный классифицировать психофизиологические показатели пользователя и адаптировать сложность VR-игры по результатам классификации.

Для достижения цели поставлены следующие задачи: изучить принципы нейровиртуальных интерфейсов и методы адаптации игр; сопоставить доступные средства реализации; определить требования к системе; спроектировать взаимодействие гарнитуры, классификатора и Unity; подготовить обучение модели и сервис предсказаний; реализовать игровую механику; рассмотреть программный код и порядок проверки работоспособности.

Объект исследования – нейровиртуальные интерактивные системы с адаптивным поведением. Предмет исследования – программные методы обработки метрик Emotiv и преобразования результата классификации в параметры игровой сложности.

В курсовой работе исследуется способ изменения сложности VR-игры по показателям электроэнцефалографической гарнитуры. Основой разработки служит игра, в которой пользователь стреляет по движущимся уткам. Метрики Emotiv передаются внешнему классификатору; полученная категория затем

влияет на настройки игры. Таким образом, показатели не заменяют обычное управление, а используются для выбора условий игрового процесса.

В работе использованы системный и сравнительный анализ, объектно-ориентированное программирование на C#, обработка данных в Python и классификация методом Random Forest. Практический результат – прототип, в котором метрики Emotiv могут влиять на параметры VR-игры через отдельный сервис предсказаний. Подобное разделение компонентов пригодно для дальнейшего развития учебных нейроадаптивных приложений.

Отчёт включает введение, две главы, заключение, список источников и приложения. Первая глава посвящена предметной области, технологиям и требованиям. Во второй описаны архитектура проекта, реализация его компонентов и порядок функциональной проверки.

1 Анализ предметной области и технологий нейровиртуальных систем

1.1 Анализ предметной области и существующих решений

В нейровиртуальной системе информация о нервной активности пользователя используется для изменения виртуальной среды. Она может влиять на управление объектами, сценарий или условия взаимодействия. Между измерением и действием системы иногда предусмотрена дополнительная обработка данных. Именно такая схема принята в рассматриваемой игре: сначала классифицируются показатели, затем выбирается уровень сложности.

По характеру взаимодействия различают активные и пассивные интерфейсы «мозг – компьютер» (BCI). Активные интерфейсы распознают паттерны, которые пользователь вызывает намеренно, например при моторном воображении. Пассивные предназначены для оценки внимания, нагрузки либо эмоциональных показателей без отдельной команды. Для игры выбран второй подход: пользователь продолжает стрелять по целям, а метрики поступают в систему адаптации независимо от его игровых действий.

Сложность стрельбы по движущимся мишеням зависит, в частности, от частоты их появления и скорости перемещения. Чем быстрее движутся цели и чем меньше пауза между их появлениями, тем меньше времени остаётся на прицеливание. Если эти параметры уменьшить, задача становится проще. Следовательно, алгоритму адаптации нужны как оценка состояния пользователя, так и конкретные игровые характеристики, которые допустимо менять.

Для сопоставления подходов рассмотрены игры с неизменной сложностью, приложения, где она зависит от игровых результатов, и системы с учётом психофизиологических показателей. Главным различием служит

источник сигнала для переключения режима. В первом случае переключения нет, во втором учитываются действия игрока, в третьем – дополнительные измерения его состояния.

Emotiv и OpenBCI предлагают средства получения нейрофизиологических данных; Unity, в свою очередь, позволяет создавать интерактивные VR-сцены. Но сочетание этих инструментов само по себе не задаёт правила изменения сложности конкретной игры. В разрабатываемом прототипе такое правило реализовано отдельно: классификатор возвращает категорию, а Unity переводит её в набор параметров игрового режима.

Таблица 1 – Сравнение подходов к адаптации игровой сложности

Критерий	**Фиксированная VR-игра**	**Адаптация по действиям**	**Адаптация по метрикам**
Источник обратной связи	Не требуется	Результаты игры	Показатели пользователя
Дополнительное оборудование	VR	VR	VR и нейрогарнитура
Индивидуальная адаптация	Ограничена	По эффективности	По оценке состояния
Основной риск	Однообразие	Неверная интерпретация ошибок	Шум и ошибки классификации

Сопоставление подходов приводит к следующему требованию: поток числовых показателей необходимо преобразовывать в понятную игровому модулю команду. Вместо передачи всех измерений непосредственно в логику целей система выдаёт одну из нескольких категорий. Поэтому сбор данных, их классификация и применение результата выделены в отдельные этапы.

1.2 Анализ методов и технологий реализации

Электроэнцефалография позволяет регистрировать электрическую активность мозга с помощью электродов на поверхности головы. В данном проекте исходные сигналы ЭЭГ непосредственно не обрабатываются. Unity получает показатели, уже рассчитанные Emotiv Cortex, а модель использует их как входные признаки. Это различие принципиально: описываемая классификация относится к метрикам Emotiv, а не к необработанным временным рядам ЭЭГ.

Набор входных признаков включает семь полей: PM.Attention.Raw, PM.Engagement.Raw, PM.Excitement.Raw, PM.Stress.Raw, PM.Relaxation.Raw, PM.Interest.Raw и PM.LongTermExcitement. Эти метрики рассматриваются как числовые признаки классификатора. Важное ограничение: сами названия признаков не доказывают, что модель определяет объективный клинический стресс; корректнее говорить об оценке выбранных меток на основе показателей Emotiv.

Имена перечисленных признаков должны совпадать на стороне Unity и сервиса предсказаний. Даже если передаются правильные значения, несоответствие названий способно нарушить формирование входного набора. Поэтому согласование полей JSON – часть проектирования интерфейса между компонентами.

Для многоклассовой классификации могут применяться логистическая регрессия, метод опорных векторов, градиентный бустинг и случайный лес. В представленном проекте выбран Random Forest. Он объединяет решения нескольких деревьев и возвращает итоговую категорию по их предсказаниям. Метод подходит для работы с используемым набором числовых признаков и тремя дискретными метками.

В задаче VR-разработки Unity обеспечивает компоненты сцены, физику столкновений и скриптовое управление объектами. XR Interaction Toolkit

предоставляет типовые механизмы взаимодействия с объектами в виртуальной реальности. Обмен между модулями может происходить через сокеты, локальные сообщения или HTTP-запросы. Для рассматриваемого прототипа HTTP REST удобен благодаря простому JSON-контракту и разделению Python и Unity на независимые процессы.

Задачи компонентов разграничены: Unity рассчитывает движение объектов, обрабатывает столкновения и отображает сцену; Python-сервис определяет категорию на основе присланных метрик. Благодаря этому результат классификации не вмешивается непосредственно в обработку выстрелов или управление объектами игроком.

HTTP-запрос не даёт мгновенного результата: требуется сформировать данные, отправить их и дождаться ответа. Поэтому реакция игры на изменение метрик происходит с некоторой задержкой. Для выбранной задачи это допустимое архитектурное разделение: классификатор меняет общий режим сложности, но не управляет положением цели по каждому отдельному измерению ЭЭГ.

1.3 Постановка задачи и функциональные требования

Разрабатываемое приложение – VR-игра со стрельбой по движущимся уткам. Одновременно с игровыми действиями клиент получает метрики Emotiv и передаёт их сервису классификации. После получения категории Unity выбирает режим сложности, от которого зависят появление целей и параметры их полёта.

Функциональные требования: подключение к доступной гарнитуре и создание сессии Cortex; подписка на поток met; сбор семи метрик; отправка JSON на локальный HTTP-адрес; получение класса low/mid/high; перевод этого класса в игровую сложность; генерация и движение мишеней; возможность стрельбы по мишеням; информирование о победе в предусмотренном сценарии.

Нефункциональные требования включают понятность интерфейса, модульность исходного кода, журналирование ошибок, воспроизводимость настройки среды и отсутствие необработанных исключений при временном отсутствии ответа сервера. Кроме того, необходимо учитывать конфиденциальность показателей пользователя и не размещать секреты авторизации в распространяемом исходном коде.

Функциональные требования описывают необходимые действия системы при штатной работе. Нефункциональные относятся к качеству взаимодействия и сопровождения: обработке ошибок соединения, понятности интерфейса и организации кода. Такое разграничение помогает проверять компоненты по отдельности.

Работа комплекса начинается с загрузки модели в сервисе классификации. Клиент Unity подключается к Emotiv Cortex и получает поток метрик. Подготовленные значения отправляются HTTP-запросом, а сервис возвращает категорию состояния. Далее GameDifficulty выбирает игровой режим; DuckSpawner и DuckFly используют его настройки при создании и перемещении целей. Так данные проходят путь от гарнитуры до изменений сцены.

Предусмотрены три игровых режима: Low, Mid и High. В принятой схеме высокий классифицированный уровень соответствует облегчению игры, низкий – её усложнению, средний – базовым параметрам. Иными словами, при более высоком уровне, обозначенном классификатором, интенсивность игровых событий уменьшается.

Вывод по первой главе. Рассмотрены принципы пассивного нейроинтерфейса и способы изменения сложности VR-игры. Для разработанного комплекса определены состав входных метрик, формат классификационного результата и требования к взаимодействию модулей. Аналитическая часть тем самым задаёт основу для проектирования программной архитектуры.

2 Проектирование и реализация адаптивной VR-игры

2.1 Выбор и обоснование средств реализации

Игровой клиент создан в Unity на C#. В коде используются Update, Physics.Raycast, компоненты сцены и события XR Interaction Toolkit. Они обеспечивают движение целей, регистрацию попаданий и действия с виртуальным оружием. Вычисление категории состояния при этом остаётся задачей отдельного Python-сервиса.

Для регистрации показателей применяется Emotiv EPOC X и программная платформа Emotiv Cortex. В компоненте EmotivMet задан адрес защищённого локального WebSocket-соединения wss://127.0.0.1:6868, инициируется сеанс и подписка на поток met. Использование штатного плагина позволяет обращаться к данным через интерфейс EmotivUnityItf, а не реализовывать протокол Cortex непосредственно внутри Unity.

Для классификации подготовлен Python-сервис с библиотеками pandas, scikit-learn и joblib. HTTP-интерфейс реализован через FastAPI и Pydantic; сервер запускается с помощью Uvicorn. Unity и сервис связаны только форматом запроса и ответа, поэтому их код можно рассматривать и изменять независимо.

В составе проекта находятся Dockerfile и docker-compose.yml для развёртывания серверной части. Если интерфейс /predict сохраняется, модель можно заменить и перезапустить сервис без повторной сборки Unity-приложения. При этом совместимость входных полей необходимо поддерживать отдельно.

Таблица 2 – Состав программного комплекса

Компонент	**Технология**	**Назначение**
VR-клиент	Unity, C#, XR Interaction Toolkit	Сцена, ввод, физика и управление игрой
Нейроданные	EPOC X, Cortex, Emotiv Unity Plugin	Получение PM-метрик
ML-сервис	Python, FastAPI, pandas	Приём данных и предсказание
Модель	scikit-learn Random Forest, joblib	Классификация на три класса
Обмен	HTTP POST, JSON	Передача метрик и ответа

2.2 Проектирование архитектуры и алгоритмов

Данные проходят следующую цепочку: Emotiv EPOC X → Emotiv Cortex (met) → EmotivMet в Unity → POST /predict → Random Forest → метка состояния → GameDifficulty → DuckSpawner и DuckFly. Последовательность отражает назначение модулей: получение показателей, вычисление категории и применение выбранного режима. Такое разделение упрощает поиск ошибок на конкретном участке обработки.

Контракт классификатора определён файлом predictor/app/main.py. Маршрут POST /predict принимает JSON с семью числовыми признаками. Из них формируется однострочный pandas.DataFrame; порядок столбцов соответствует тому, который использовался при подготовке модели. После вызова предсказания сервис возвращает категорию состояния. Согласованность имён и последовательности признаков здесь необходима для корректной передачи данных.

В EmotivMet.cs поля, поступающие от Emotiv, преобразуются по словарю: attention, eng, exc, str, rel, int, lex – в полные имена колонок классификатора. После накопления трёх обработанных образцов выполняется SendPredictionRequest. Значение logInterval по умолчанию составляет одну

секунду, следовательно, при стабильном поступлении данных запрос выполняется ориентировочно через три цикла обработки.

Словарь соответствия имён в EmotivMet связывает короткие обозначения Cortex с названиями признаков модели. Клиент приводит полученные поля к ожидаемому формату до обращения к /predict. Поэтому проверять корректность обмена следует ещё до того, как оценивать реакцию игрового сценария.

GameDifficulty.cs сопоставляет метку состояния с игровым режимом по правилу low (низкий стресс) → High (сложность выше), mid (среднее состояние) → Mid (средняя сложность), high (высокий стресс) → Low (сложность ниже). По умолчанию задан режим Mid. Такая инверсия означает, что низкий классифицированный уровень приводит к более сложной игре, а высокий – к более простой. При изменении значения вызывается событие, на которое ориентируются другие игровые компоненты.

За создание шаблона уток отвечает DuckSpawner, который определяет интервал появления на основе базового spawnInterval = 2 с. Для Low он увеличивается в 1,5 раза, для Mid остаётся равным базовому, для High уменьшается вдвое. Экземпляру DuckFly передаётся текущая сложность, чтобы параметры движения устанавливались при инициализации объекта. Изменение уровня в первую очередь влияет на вновь созданные объекты.

Интервал создания новых уток – не единственный изменяемый параметр. Сложность также определяется скоростью и особенностями траектории. Вместе эти настройки позволяют задавать различающееся поведение целей, сохраняя прежнюю задачу игрока – обнаружить мишень и выполнить выстрел.

Таблица 3 – Параметры сложности по исходному коду

Сложность	Интервал при базе 2 с	Скорость утки	Время жизни
Low	3 с	1,5–2,5 ед./с	14 с
Mid	2 с	2,5–4 ед./с	12 с
High	1 с	5–7 ед./с	10 с

Дополнительно движение описывается синусоидальным отклонением по вертикали. Для Low амплитуда равна 0,5, а частотный параметр 1,5; для Mid – 0,4 и 2; для High – 0,2 и 4. В исходном коде координата по оси Y задаётся выражением $y = y_0 + A \cdot \sin(t \cdot f)$, а поступательная координата изменяется пропорционально скорости и Time.deltaTime. Это позволяет варьировать динамику целей без изменения базовой игровой механики.

Обмен между компонентами содержит потенциальные точки отказа: сообщение Cortex может не поступить, а HTTP-сервис – не ответить на запрос. Код EmotivMet предусматривает регистрацию сообщений и проверку авторизации. Обработка остальных сбоев должна оцениваться при практическом запуске. По одному исходному коду нельзя сделать вывод об устойчивости всей связки с реальной гарнитурой.

2.3 Программная реализация и проверка функционирования

Программный комплекс нейроадаптивной VR-игры состоит из игровых компонентов Unity на языке C# и сервиса классификации на Python. Их взаимодействие обеспечивает получение метрик Emotiv, определение категории состояния пользователя и изменение сложности игрового процесса. Передача данных между компонентами осуществляется посредством HTTP-запросов в формате JSON.

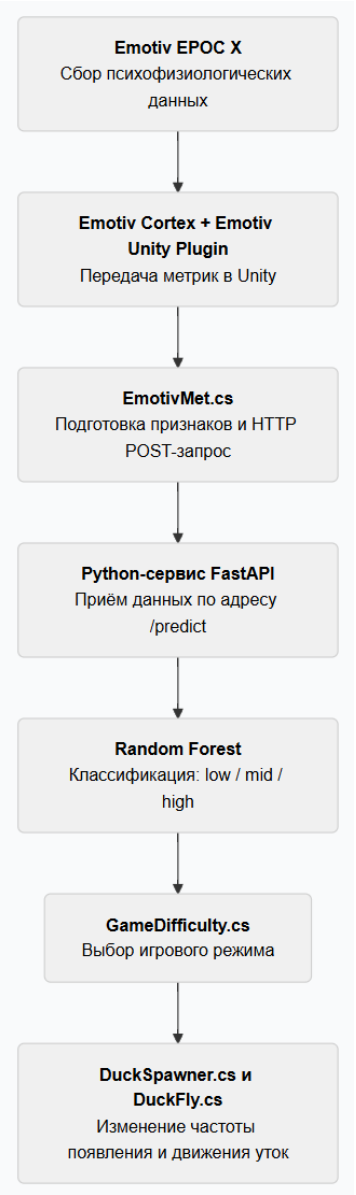


Рисунок 2.1 — Информационный поток нейроадаптивной VR-игры

Получение психофизиологических показателей реализовано в компоненте EmotivMet.cs, взаимодействующем с Emotiv Cortex через Emotiv Unity Plugin. После подписки на поток met система получает семь показателей: внимание, вовлечённость, возбуждение, стресс, расслабление, интерес и долгосрочное возбуждение. После накопления трёх обработанных образцов формируется запрос к сервису. Данные передаются методом POST в виде JSON-объекта с числовыми значениями признаков. Пример запроса представлен на рисунке 2.2.



Рисунок 2.2 – Пример POST запроса к сервису классификации.

Сервис классификации реализован с использованием FastAPI, pandas и scikit-learn. При запуске загружается сохранённая модель Random Forest, которая обрабатывает входные признаки и определяет одну из трёх категорий: low, mid или high. Результат возвращается клиенту Unity в формате JSON. Оценка матрицы ошибок классификации алгоритма представлена на рисунке 2.3. Таким образом, классификация выполняется независимо от игровой сцены, что упрощает модификацию модели и серверной части.

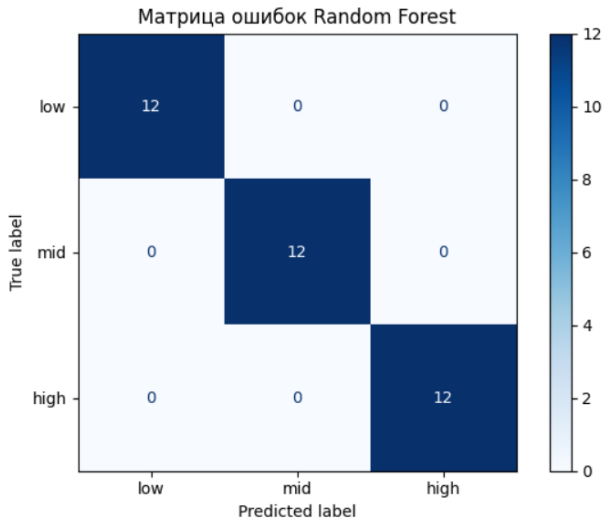


Рисунок 2.3 – Матрица ошибок классификатора

Преобразование результата классификации в игровой режим выполняет компонент GameDifficulty.cs. Метке low соответствует режим High, метке mid – Mid, а метке high – Low. До получения результата используется базовый режим Mid. Такая зависимость позволяет изменять интенсивность игрового процесса в соответствии с классифицированной категорией состояния пользователя.

Создание мишеней выполняет DuckSpawner.cs, регулирующий интервал появления уток. При базовом значении spawnInterval = 2 с для режима Low интервал составляет 3 с, для Mid – 2 с, для High – 1 с. Скрипт DuckFly.cs определяет скорость и траекторию движения объектов. Дополнительно используется синусоидальное отклонение по вертикали:

$$y = y_0 + A \cdot \sin(t \cdot f)$$

где (y_0) – исходная координата, (A) – амплитуда колебаний, (t) – время, (f) – частотный параметр. Скорость уток составляет 1,5–2,5 ед./с для Low, 2,5–4 ед./с для Mid и 5–7 ед./с для High. Таким образом, сложность определяется частотой появления целей и особенностями их движения.



Рисунок 2.4 – Игровая сцена VR-приложения.

Взаимодействие с виртуальным оружием реализовано в Shotgun.cs с использованием XRGrabInteractable. При выстреле выполняется проверка попадания физическим лучом, а при обнаружении утки с компонентом DuckFly обрабатывается попадание. Скрипт DuckShooter.cs дополнительно предусматривает стрельбу с помощью мыши, подсчёт попаданий и завершение игрового сценария при достижении десяти очков. Эти механизмы относятся к разным сценариям управления, поэтому корректность подсчёта очков зависит от используемого способа стрельбы.

Для проверки функционирования комплекса предусмотрены проверки передачи данных, работы классификатора, переключения режимов и изменения поведения игровых объектов. Основные проверяемые операции представлены в таблице 4.

Таблица 4 –Программа функциональной проверки

Проверка	**Действие**	**Ожидаемый результат**
Доступность сервиса	GET /health после загрузки модели	HTTP 200, status: ok
Формат ответа	POST /predict с семью метриками	level и probabilities
Инверсия стресса	Подать low / mid / high	High / Mid / Low
Появление уток	Переключить сложность	Интервал 1 / 2 / 3 с
VR-выстрел	Активировать Shotgun	Удаление поражённой утки
Завершение по счёту	Набрать 10 попаданий через DuckShooter	Панель победы; отключение спавнера
Потеря соединения	Остановить ML-сервис	Сообщение об HTTP-ошибке

В ходе функционального тестирования были проверены основные компоненты разработанного программного комплекса. Проверка HTTP-интерфейса подтвердила доступность сервиса классификации и корректность формирования JSON-ответов. При передаче различных категорий состояния

пользователя установлено соответствие между уровнем стресса и игровой сложностью. Также проверена работа механизмов появления и движения уток, обработки попаданий и завершения игрового сценария.

Разработана архитектура программного комплекса, объединяющая Unity, Emotiv Cortex и Python-сервис классификации. Определены правила преобразования классификационных меток в уровни сложности и рассмотрены механизмы изменения параметров игровых объектов. Сформированы критерии проверки основных компонентов, позволяющие оценивать корректность их взаимодействия и дальнейшую работоспособность комплекса.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была рассмотрена задача разработки нейроадаптивной VR-игры, в которой изменение сложности игрового процесса осуществляется с учётом психофизиологических показателей пользователя. Актуальность данного направления обусловлена возможностью создания интерактивных виртуальных сред, адаптирующихся к индивидуальным особенностям пользователя и обеспечивающих более гибкое управление условиями взаимодействия.

В первой главе проведён анализ принципов функционирования нейровиртуальных интерфейсов и существующих подходов к адаптации сложности интерактивных приложений. Рассмотрены особенности использования электроэнцефалографических гарнитур для получения психофизиологических показателей, а также возможности применения методов машинного обучения для классификации состояния пользователя. На основе проведённого анализа сформулированы функциональные и нефункциональные требования к разрабатываемому программному комплексу.

Во второй главе спроектирована архитектура нейроадаптивной VR-игры, предусматривающая взаимодействие игровой среды Unity, гарнитуры Emotiv EPOC X, программной платформы Emotiv Cortex и отдельного сервиса классификации на Python. Для обработки входных данных предусмотрено использование алгоритма Random Forest, определяющего одну из трёх категорий состояния пользователя: низкий, средний или высокий стресс. Передача показателей и результатов классификации между компонентами организована посредством HTTP-запросов с использованием формата JSON. Полученная категория преобразуется в соответствующий игровой режим, влияющий на частоту появления движущихся мишеней, скорость их перемещения и характер движения.

В результате проектирования и программной реализации подготовлена структура программного комплекса, объединяющая получение психофизиологических показателей, их классификацию и изменение параметров виртуальной среды. Реализованные компоненты позволяют разделить задачи обработки данных и управления игровой механикой, что упрощает дальнейшее развитие и модификацию системы. При этом пользователь сохраняет непосредственное управление игровыми действиями, а классификационные результаты используются для изменения условий прохождения игры.

Таким образом, поставленная цель курсовой работы – разработка программного комплекса для классификации психофизиологических показателей и адаптации сложности VR-игры – определила выбор архитектуры, технологий и алгоритмов реализации. Полученные результаты формируют основу для дальнейшего совершенствования нейроадаптивной игровой среды, проведения экспериментальных исследований и оценки эффективности применения методов машинного обучения при адаптации виртуальной реальности к состоянию пользователя.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Айвазян С.А. Прикладная статистика: основы моделирования и первичная обработка данных. – М. : Финансы и статистика, 1983. – 167 с.

2. Бестаева В.В. Использование нейроинтерфейса Emotiv Insight для управления роботизированными устройствами // В сборнике: Межвузовская научно-техническая конференция студентов, аспирантов и молодых специалистов имени Е.В. Арменского. Материалы конференции. – М., 2021. – С. 110–112.

3. Бонд Д.Г. Unity и C#. Геймдев от идеи до реализации. – СПб. : Питер, 2019. – 928 с.

4. Васильев А.Н., Тархов Д.А. Принципы и техника нейросетевого моделирования. – М. : Гостехиздат, 2015. – 334 с.

5. Васильева Л.А., Тананайко С.О. Опыт применения нейрокомпьютерного интерфейса Emotiv Epoc для лингвистических исследований // Научный альманах. – 2016. – № 1-3 (15). – С. 402–410.

6. Возненко Т.И., Петрова А.И. Модуль тестирования команд управления мобильным робототехническим комплексом с помощью нейрокомпьютерного интерфейса Emotiv : свидетельство о регистрации программы для ЭВМ RU 2021660702, 30.06.2021. Заявка № 2021619578 от 18.06.2021.

7. Гончаров С.М., Маркин М.Е. «Интерфейс мозг-компьютер» как нестандартная технология управления и передачи информации // Доклады Томского государственного университета систем управления и радиоэлектроники. – 2014. – № 2 (32). – С. 252–256.

8. Григорян Р.К. Гибридный интерфейс мозг-компьютер на основе анализа электроэнцефалограммы : свидетельство о регистрации программы для ЭВМ RU 2019615355, 24.04.2019. Заявка № 2019614149 от 10.04.2019.

9. Жерон О. Прикладное машинное обучение с помощью Scikit-Learn и TensorFlow: концепции, инструменты и техники для создания интеллектуальных систем. – СПб. : Альфа-книга, 2018. – 688 с.
10. Корнилов А.В. UNITY. Полное руководство. – СПб. : Наука и техника, 2020. – 432 с.
11. Лутц М. Программирование на Python. Том 1. – СПб. : 4-е изд., 2011. – 992 с.
12. Лутц М. Программирование на Python. Том 2. – М. : Символ-плюс, 2013. – 334 с.
13. Мюллер А. Введение в машинное обучение с помощью Python. – М. : Диалектика, 2017. – 480 с.
14. Силен Д., Мейсман А. Основы Data Science и Big Data. Python наука о данных. – СПб. : Питер, 2017. – 336 с.
15. Торн А., Раттц В., Джозеф С. Основы анимации в Unity. – М. : Наука и техника, 2016. – 175 с.
16. Уоссермен Ф. Нейрокомпьютерная техника: теория и практика. – М. : Мир, 1992. – 236 с.
17. Уссаева А.Г., Жумманов У.В., Аннаев А.А., Бориева А.Г., Гапбаров А.С. Brain-computer interface based brain wave and neural signal detector // Молодой ученый. – 2024. – № 44 (543). – С. 27–29.
18. Форсье Д., Биссекс П. Django. Разработка веб-приложений на Python. – М. : Символ-Плюс, 2021. – 323 с.
19. Хокинг Д.С. Unity в действии. Мультиплатформенная разработка на C#. – СПб. : Питер, 2019. – 352 с.
20. Храмов А.Е. Создание интерфейсов мозг-компьютер: новая междисциплинарная задача на стыке нейрофизиологии, прикладной математики и нелинейной науки // В сборнике: Экология: синтез естественно-научного, технического и гуманитарного знания. – 2012. – С. 333–335.

Приложение А. Код сервиса классификации

```
import

logger = logging.getLogger("uvicorn.error")

FEATURES = [
    "PM.Attention.Raw",
    "PM.Engagement.Raw",
    "PM.Excitement.Raw",
    "PM.Stress.Raw",
    "PM.Relaxation.Raw",
    "PM.Interest.Raw",
    "PM.LongTermExcitement",
]
MODEL_PATH = Path(os.environ.get("MODEL_PATH", "/app/rf_active_model.joblib"))
model = None
class PredictRequest(BaseModel):
    model_config = ConfigDict(populate_by_name=True)
    attention: Annotated[float | None, Field(alias="PM.Attention.Raw")] = None
    engagement: Annotated[float | None, Field(alias="PM.Engagement.Raw")] = None
    excitement: Annotated[float | None, Field(alias="PM.Excitement.Raw")] = None
    stress: Annotated[float | None, Field(alias="PM.Stress.Raw")] = None
    relaxation: Annotated[float | None, Field(alias="PM.Relaxation.Raw")] = None
    interest: Annotated[float | None, Field(alias="PM.Interest.Raw")] = None
    long_term_excitement: Annotated[float | None, Field(alias="PM.LongTermExcitement")] = None
class PredictResponse(BaseModel):
    level: str
    probabilities: dict[str, float]
@asynccontextmanager
async def lifespan(app: FastAPI):
    global model
    if not MODEL_PATH.is_file():
        logger.error(f"Файл модели не найден: {MODEL_PATH}")
        raise FileNotFoundError(f"Model file not found: {MODEL_PATH}")
    model = joblib.load(MODEL_PATH)
    logger.info(f"Модель успешно загружена из {MODEL_PATH}")
    yield
    model = None
    logger.info("Модель выгружена из памяти")
app = FastAPI(title="Random Forest predictor", lifespan=lifespan)
@app.get("/health")
def health():
    if model is None:
        raise HTTPException(status_code=503, detail="model not loaded")
    return {"status": "ok"}
@app.post("/predict", response_model=PredictResponse)
def predict(payload: PredictRequest):
    received_data = payload.model_dump(by_alias=True)
    logger.info(f"Получен запрос на предсказание: {received_data}")

    try:
        frame = pd.DataFrame([received_data], columns=FEATURES)
        level = model.predict(frame)[0]
        proba = model.predict_proba(frame)[0]

        probabilities = {
            str(label): float(value) for label, value in zip(model.classes_, proba)
        }

        # Логируем результат
        logger.info(f"Результат предсказания: level='{level}', probabilities={probabilities}")

        return PredictResponse(level=str(level), probabilities=probabilities)

    except Exception as e:
        # Логируем ошибку, если что-то пошло не так во время инференса
        logger.error(f"Ошибка при обработке запроса: {e}")
        raise HTTPException(status_code=500, detail="Internal Server Error")
```

Приложение Б. Код скрипта Unity

```
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Networking;
using EmotivUnityPlugin;
public class EmotivMet : MonoBehaviour
{
    private EmotivUnityItf emotiv;

    private bool querySent;
    private bool sessionRequested;
    private bool subscriptionRequested;
    private float lastStatusLogTime;
    private float lastDataLogTime;
    private float lastQueryTime;
    private float sessionRequestTime;
    private int sessionAttempts;
    private const int MaxSessionAttempts = 3;
    private const float QueryRetryInterval = 5f;
    private const float SessionTimeout = 10f;

    [Header("Cortex credentials")]
    public string clientId = "YOUR_CLIENT_ID";
    public string clientSecret = "YOUR_CLIENT_SECRET";

    [Header("Cortex")]
    public string appName = "MyUnityApp";
    public string cortexUrl = "wss://127.0.0.1:6868";

    [Header("Headset")]
    public string headsetId = "EPOCX-F416A4AD";

    [Header("MET logging")]
    [Min(0.1f)]
    public float logInterval = 1f;

    [Header("ML Predictor")]
    public string predictorUrl = "http://127.0.0.1:8000/predict";
    private int metSampleCounter = 0;
    private const int PredictEveryNSamples = 3; // Отправлять каждый 3-й сэмпл

    [Serializable]
    public class PredictResponse
    {
        public string level;
        // probabilities можно добавить сюда же, если понадобятся
    }
    private void Start()
    {
        emotiv = EmotivUnityItf.Instance;

        if (emotiv == null)
        {
            Debug.LogError("[Emotiv] EmotivUnityItf.Instance не найден");
            enabled = false;
            return;
        }
        if (string.IsNullOrWhiteSpace(clientId) ||
            string.IsNullOrWhiteSpace(clientSecret) ||
            string.IsNullOrWhiteSpace(appName) ||
            clientId == "YOUR_CLIENT_ID" ||
            clientSecret == "YOUR_CLIENT_SECRET")
        {
            Debug.LogError("[Emotiv] Укажи clientId, clientSecret и appName");
            enabled = false;
            return;
        }
        emotiv.ErrorMsgReceived += OnCortexError;

        try
        {
            emotiv.Init(clientId, clientSecret, appName, false, true, cortexUrl);
```

```
            emotiv.Start();
        }
        catch (Exception e)
        {
            Debug.LogError($"[Emotiv] Ошибка запуска: {e.Message}");
        }
    }

    private void Update()
    {
        if (emotiv == null) return;

        if (!emotiv.IsAuthorizedOK)
        {
            LogStatus();
            return;
        }

        if (!querySent)
        {
            QueryHeadsets();
            LogStatus();
            return;
        }

        if (emotiv.IsSessionCreated)
        {
            if (!subscriptionRequested)
                SubscribeToMET();

            if (subscriptionRequested)
                LogMET();
        }
        else
        {
            if (sessionRequested && Time.time - sessionRequestTime >= SessionTimeout)
            {
                Debug.LogWarning($"[Emotiv] Таймаут создания сессии. Попытка {sessionAttempts}/{MaxSessionAttempts}");
                sessionRequested = false;
            }

            if (!sessionRequested && sessionAttempts < MaxSessionAttempts)
            {
                TryCreateSession();
            }
        }
        LogStatus();
    }
    private void QueryHeadsets()
    {
        if (Time.time - lastQueryTime < QueryRetryInterval && lastQueryTime > 0f)
            return;
        lastQueryTime = Time.time;
        try
        {
            emotiv.QueryHeadsets();
            querySent = true;
        }
        catch (Exception e)
        {
            Debug.LogError($"[Emotiv] QueryHeadsets: {e.Message}");
            querySent = false;
        }
    }

    private void TryCreateSession()
    {
        try
        {
            List<Headset> detected = emotiv.GetDetectedHeadsets();
            if (detected == null || detected.Count == 0)
            {
                if (Time.time - lastQueryTime >= QueryRetryInterval)
                    querySent = false;
                return;
```

```
        }

        if (string.IsNullOrWhiteSpace(headsetId))
        {
            Debug.LogError("[Emotiv] Не указан headsetId");
            enabled = false;
            return;
        }
        sessionAttempts++;
        sessionRequested = true;
        sessionRequestTime = Time.time;
        emotiv.CreateSessionWithHeadset(headsetId);
    }
    catch (Exception e)
    {
        sessionRequested = false;
        Debug.LogError($"[Emotiv] Ошибка создания сессии: {e.Message}");
    }
}
private void SubscribeToMET()
{
    if (subscriptionRequested || !emotiv.IsSessionCreated) return;
    try
    {
        emotiv.DataSubLog = "";
        emotiv.SubscribeData(new List<string> { "met" });
        subscriptionRequested = true;
    }
    catch (Exception e)
    {
        Debug.LogError($"[Emotiv] Ошибка подписки на MET: {e.Message}");
    }
}
private void LogMET()
{
    if (Time.time - lastDataLogTime < logInterval) return;
    lastDataLogTime = Time.time;

    try
    {
        if (emotiv.GetNumberPMSamples() <= 0) return;
        List<string> labels = emotiv.GetPMLists();
        if (labels == null || labels.Count == 0) return;
        List<string> values = new List<string>();
        Dictionary<string, double> metDataDict = new Dictionary<string, double>();
        foreach (string label in labels)
        {
            double value = emotiv.GetPMData(label);
            values.Add($"{label}={value:F4}");
            metDataDict[label] = value;
        }
        Debug.Log($"[Emotiv][MET] {string.Join("; ", values)}");

        metSampleCounter++;
        if (metSampleCounter >= PredictEveryNSamples)
        {
            metSampleCounter = 0;
            StartCoroutine(SendPredictionRequest(metDataDict));
        }
    }
    catch (Exception e)
    {
        Debug.LogWarning($"[Emotiv][MET] Ошибка чтения: {e.Message}");
    }
}

private IEnumerator SendPredictionRequest(Dictionary<string, double> metDataDict)
{
    // 1. Маппинг ключей Emotiv в ключи FastAPI
    Dictionary<string, string> keyMapping = new Dictionary<string, string>
    {
        { "attention", "PM.Attention.Raw" },
        { "eng",       "PM.Engagement.Raw" },
        { "exc",       "PM.Excitement.Raw" },
        { "str",       "PM.Stress.Raw" },
```

```
            { "rel",       "PM.Relaxation.Raw" },
            { "int",       "PM.Interest.Raw" },
            { "lex",       "PM.LongTermExcitement" }
        };

        string jsonPayload = "{";
        bool first = true;
        foreach (var kvp in metDataDict)
        {
            string apiKey = keyMapping.ContainsKey(kvp.Key) ? keyMapping[kvp.Key] : kvp.Key;

            if (!first) jsonPayload += ",";
            jsonPayload += $"\"{apiKey}\": {kvp.Value.ToString(System.Globalization.CultureInfo.InvariantCulture)}";
            first = false;
        }
        jsonPayload += "}";
        using (UnityWebRequest request = new UnityWebRequest(predictorUrl, "POST"))
        {
            byte[] bodyRaw = System.Text.Encoding.UTF8.GetBytes(jsonPayload);
            request.uploadHandler = new UploadHandlerRaw(bodyRaw);
            request.downloadHandler = new DownloadHandlerBuffer();
            request.SetRequestHeader("Content-Type", "application/json");
            yield return request.SendWebRequest();
            if (request.result == UnityWebRequest.Result.Success)
            {
                string responseJson = request.downloadHandler.text;
                try
                {
                    PredictResponse response = JsonUtility.FromJson<PredictResponse>(responseJson);

                    if (response != null && !string.IsNullOrEmpty(response.level))
                    {
                        Debug.Log($"[ML Predict] Ответ сервера: {response.level}");

                        // >>> Единственное смысловое изменение: сообщаем игре новый уровень сложности.
                        GameDifficulty.SetFromStressLevel(response.level);
                    }
                }
                catch (Exception e)
                {
                    Debug.LogError($"[ML Predict] Ошибка парсинга ответа: {e.Message}");
                }
            }
            else
            {
                Debug.LogError($"[ML Predict] Ошибка HTTP запроса: {request.error}");
            }
        }
    }
    private void LogStatus()
    {
        if (Time.time - lastStatusLogTime < 5f) return;
        lastStatusLogTime = Time.time;
    }

    private void OnCortexError(object sender, ErrorMsgEventArgs error)
    {
        Debug.LogError($"[Emotiv][Cortex Error] Code={error.Code}; Method={error.MethodName}; Message={error.MessageError}");
    }
    private void OnApplicationQuit()
    {
        if (emotiv == null) return;

        try
        {
            emotiv.ErrorMsgReceived -= OnCortexError;
            emotiv.Stop();
        }
        catch (Exception e)
        {
            Debug.LogWarning($"[Emotiv] Ошибка остановки: {e.Message}");
        }
    }

}
```